KIT – KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY

COIMBATORE – 641 402

APPROVED BY AICTE AFFILIATED TO ANNA UNIVERSITY, CHENNAI
ACCREDITED WITH 'A' GRADE BY NAAC & NBA

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Course Code / Name: B25ADT303 / OBJECT ORIENTED PROGRAMMING USING JAVA
Year / Semester: II / III • Academic Year: 2026

Team Members / Student Details:

SI. NO	NAME	REGISTER NO
1.	PRINCE S	711525BAM034

S.NO	MINI PROJECT TITLE
1.	FoodFlow: Online Food Order Processing Simulator using Multi-Threaded Concurrency and Neobrutalism UI Architecture

Evaluation Rubrics & Marks Allotment:

Technical Content (20)	Completeness (10)	Originality (5)	Timely Submission (5)	Total (40)

Signature of the Student **Signature of the Faculty In-Charge** **Head of the Department (AI & ML)**

KIT – KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY
COIMBATORE – 641 402
APPROVED BY AICTE AFFILIATED TO ANNA UNIVERSITY, CHENNAI
ACCREDITED WITH 'A' GRADE BY NAAC & NBA

1. Title of the Project

FoodFlow: Online Food Order Processing Simulator using Multi-Threaded Concurrency and Neobrutalism UI Architecture

2. Aim

To design and implement a multi-threaded Online Food Order Processing Simulator in Java that visualizes real-time order lifecycle transitions (Preparation → Packaging → Delivery) using concurrent worker threads, thread-safe PriorityBlockingQueues, and a high-contrast Neobrutalism user interface without UI freezing.

3. Problem Statement

Modern cloud kitchens and online food delivery dispatchers handle high volumes of concurrent orders. Single-threaded or poorly synchronized architectures suffer from critical shortcomings:

- **Pipeline Blocking:** When food preparation takes 2–5 seconds, sequential processing blocks subsequent orders from packaging or delivery dispatch.
- **UI Freezing:** Running heavy backend execution or sleeps directly on the UI thread freezes graphical user interfaces.
- **Lack of Prioritization:** High-priority or VIP orders get stuck behind older regular orders in standard FIFO queues.
- **Race Conditions:** Without proper thread-safe synchronization, shared order collections produce data corruption, deadlocks, and missed updates.

An asynchronous, multi-threaded pipeline with decoupled producer-consumer queue handoffs is required to coordinate distinct workers concurrently while reflecting immediate state changes in a responsive interface.

4. Objective

- To develop three dedicated worker threads: `KitchenWorker` (simulated cooking), `PackagingWorker` (sealing & boxing), and `DeliveryWorker` (courier dispatch).
- To implement thread-safe producer-consumer queue handoffs using Java's `PriorityBlockingQueue<Order>`.
- To implement natural priority sorting via the `Comparable<Order>` interface so VIP orders jump ahead of standard orders.
- To construct a high-contrast Neobrutalism GUI (Swing) with real-time linear progress bars, 3-lane visual board, atomic live telemetry, and synchronized logging.
- To measure execution duration per stage with millisecond precision and provide one-click CSV audit log export.

5. Algorithm

Step 1 (System Initialization): Initialize the `OrderProcessingSystem` orchestrator. Create three thread-safe `PriorityBlockingQueue<Order>` instances: `kitchenQueue`, `packagingQueue`, and `deliveryQueue`. Launch an `ExecutorService` fixed thread pool managing three named workers.

Step 2 (Order Ingestion): When an order is submitted (manually or via batch simulator), generate a unique Order ID (e.g., `ORD-1001`), instantiate an encapsulated `Order` object with customer details, items, VIP flag, and submission timestamp. Enqueue the order into `kitchenQueue` and notify the GUI Event Dispatch Thread.

Step 3 (Kitchen Preparation): `KitchenWorker` polls `kitchenQueue`. Upon acquiring an order, it updates status to `PREPARING`, calculates simulated duration (2.0s – 5.0s, expedited for VIP), executes in 100ms progress ticks, updates status to `PREPARED`, and puts the order into `packagingQueue`.

Step 4 (Packaging & Boxing): `PackagingWorker` polls `packagingQueue`, transitions status to `PACKAGING`, simulates boxing (1.0s – 3.0s), transitions status to `PACKAGED`, and enqueues to `deliveryQueue`.

Step 5 (Delivery Transit): `DeliveryWorker` polls `deliveryQueue`, transitions status to `DELIVERING`, simulates courier transit (1.0s – 2.0s), transitions status to `DELIVERED`, records timestamps, plays a synthesized completion chime, and moves order to history.

KIT – KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY
COIMBATORE – 641 402
APPROVED BY AICTE AFFILIATED TO ANNA UNIVERSITY, CHENNAI
ACCREDITED WITH 'A' GRADE BY NAAC & NBA

Order Lifecycle State Transitions & Action Matrix:

Status	Stage	Active Worker	Queue Operation	UI Color	Simulated Time
PENDING	Kitchen	Main / Dispatcher	`offer(Order)`	Grey	Immediate
PREPARING	Kitchen	KitchenWorker-1	`poll(timeout)`	Electric Blue	2.0s – 5.0s
PREPARED	Kitchen	KitchenWorker-1	`put(packagingQueue)`	Cyan	Instant Handoff
PACKAGING	Packaging	PackagingWorker-1	`poll(timeout)`	Neon Yellow	1.0s – 3.0s
PACKAGED	Packaging	PackagingWorker-1	`put(deliveryQueue)`	Light Yellow	Instant Handoff
DELIVERING	Delivery	DeliveryWorker-1	`poll(timeout)`	Acid Green	1.0s – 2.0s
DELIVERED	Terminal	DeliveryWorker-1	Archive to History	Bright Green	Audio Chime

6. Workflow Diagram (Flowchart)

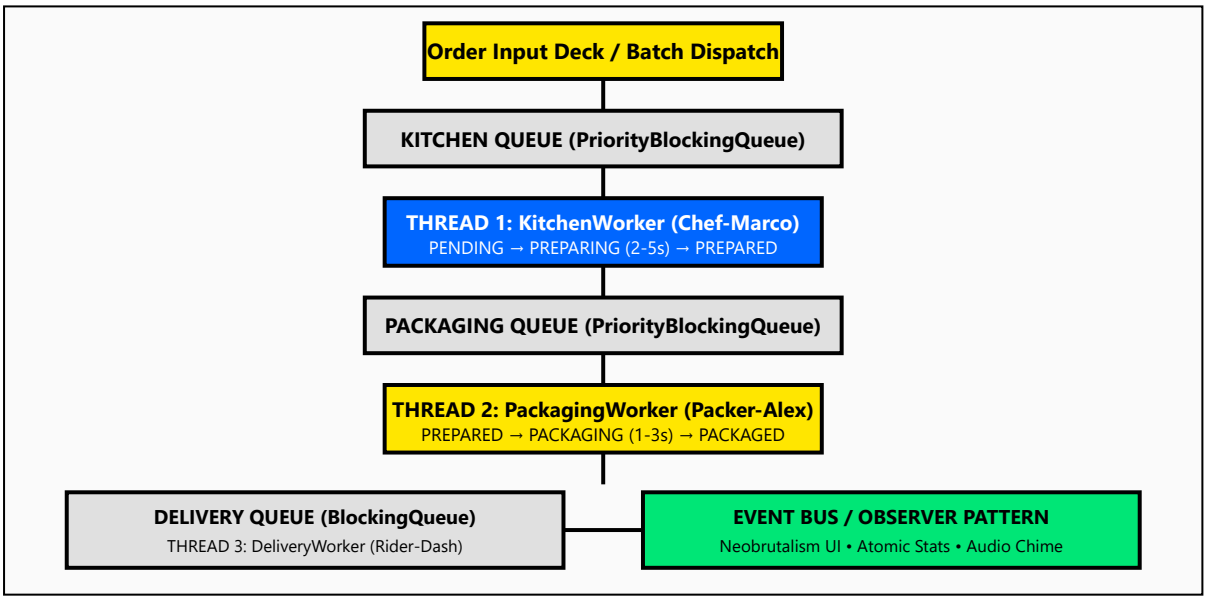


Fig 1: Concurrent Producer-Consumer Multithreaded Pipeline Architecture

KIT – KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY
COIMBATORE – 641 402
APPROVED BY AICTE AFFILIATED TO ANNA UNIVERSITY, CHENNAI
ACCREDITED WITH 'A' GRADE BY NAAC & NBA

7. Java Program Source Code (OOP Encapsulation & Models)

models/OrderStatus.java (State Machine with Predicates & Hex Color Tokens):

```
package models;
public enum OrderStatus {
    PENDING("PENDING", "Waiting in Kitchen Queue", "#E0E0E0"),
    PREPARING("PREPARING", "Chef cooking food", "#0066FF"),
    PREPARED("PREPARED", "Ready for packaging", "#66B2FF"),
    PACKAGING("PACKAGING", "Boxing items", "#FFE600"),
    PACKAGED("PACKAGED", "Ready for courier", "#FFE866"),
    DELIVERING("DELIVERING", "Courier in transit", "#00E676"),
    DELIVERED("DELIVERED", "Delivered successfully", "#00FF66"),
    CANCELLED("CANCELLED", "Order cancelled", "#FF3366");

    private final String displayName, description, hexColor;
    OrderStatus(String name, String desc, String color) {
        this.displayName = name; this.description = desc; this.hexColor = color;
    }
    public String getDisplayName() { return displayName; }
    public String getHexColor() { return hexColor; }
    public boolean isInKitchen() { return this == PENDING || this == PREPARING || this == PREPARED; }
    public boolean isInPackaging() { return this == PACKAGING || this == PACKAGED; }
    public boolean isInDelivery() { return this == DELIVERING; }
    public boolean isTerminal() { return this == DELIVERED || this == CANCELLED; }
}
```

models/Order.java (Encapsulation, Defensive Copying & Priority Sorting):

```
package models;
import java.util.*;

public class Order implements Comparable<Order> {
    private final String orderId;
    private final String customerName;
    private final List<String> items;
    private final long createdTimestamp;
    private final boolean isVip;

    private volatile OrderStatus status;
    private volatile long completedTimestamp;
    private volatile long stageStartTimestamp;
    private volatile long estimatedDurationMs;
    private volatile String currentWorkerName = "Unassigned";

    public Order(String id, String customer, List<String> items, boolean isVip) {
        this.orderId = Objects.requireNonNull(id);
        this.customerName = Objects.requireNonNull(customer);
        this.items = items != null ? new ArrayList<>(items) : new ArrayList<>(); // Defensive Copy
        this.isVip = isVip;
        this.createdTimestamp = System.currentTimeMillis();
        this.stageStartTimestamp = this.createdTimestamp;
        this.status = OrderStatus.PENDING;
    }

    public synchronized void setStatus(OrderStatus newStatus) {
        this.status = newStatus;
        this.stageStartTimestamp = System.currentTimeMillis();
        if (newStatus.isTerminal()) this.completedTimestamp = this.stageStartTimestamp;
    }

    // Natural Ordering: VIP orders jump to front of priority queue
    @Override
    public int compareTo(Order other) {
        if (this.isVip != other.isVip) return this.isVip ? -1 : 1;
        return Long.compare(this.createdTimestamp, other.createdTimestamp);
```

KIT – KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY
COIMBATORE – 641 402
APPROVED BY AICTE AFFILIATED TO ANNA UNIVERSITY, CHENNAI
ACCREDITED WITH 'A' GRADE BY NAAC & NBA

7. Java Program Source Code (Threading Architecture & Coordination)

threading/WorkerThread.java (Worker Contract Interface):

```
package threading;
import models.Order;

public interface WorkerThread extends Runnable {
    String getWorkerName();
    String getThreadState();
    int getOrdersProcessed();
    Order getCurrentOrder();
    void pauseWorker();
    void resumeWorker();
    boolean isPaused();
    void shutdown();
}
```

threading/KitchenWorker.java (Simulated Preparation & Queue Handoff):

```
package threading;
import core.OrderProcessingSystem;
import models.*;
import java.util.concurrent.*;

public class KitchenWorker implements WorkerThread {
    private final String workerName;
    private final BlockingQueue<Order> inputQueue, outputQueue;
    private final OrderProcessingSystem system;
    private volatile boolean running = true, paused = false;
    private final Object pauseLock = new Object();
    private volatile int ordersProcessed = 0;

    public KitchenWorker(String name, BlockingQueue<Order> inQ, BlockingQueue<Order> outQ, OrderProcessing
System sys) {
        this.workerName = name; this.inputQueue = inQ; this.outputQueue = outQ; this.system = sys;
    }

    @Override
    public void run() {
        while (running && !Thread.currentThread().isInterrupted()) {
            try {
                synchronized (pauseLock) {
                    while (paused && running) pauseLock.wait();
                }
                Order order = inputQueue.poll(500, TimeUnit.MILLISECONDS);
                if (order == null) continue;

                // Pick up order: PENDING -> PREPARING
                order.setCurrentWorkerName(workerName);
                int prepTimeMs = order.isVip() ? 2200 : 3500;
                order.setEstimatedDurationMs(prepTimeMs);
                system.updateOrderStatus(order, OrderStatus.PREPARING);

                // Simulate cooking in 100ms ticks for UI animation
                long start = System.currentTimeMillis();
                while (System.currentTimeMillis() - start < prepTimeMs && running) {
                    Thread.sleep(Math.min(100, prepTimeMs - (System.currentTimeMillis() - start)));
                }

                // Stage completion: PREPARING -> PREPARED -> Handoff
                order.setKitchenDurationMs(System.currentTimeMillis() - start);
                system.updateOrderStatus(order, OrderStatus.PREPARED);
                ordersProcessed++;
                outputQueue.put(order); // Non-blocking handoff to Packaging
            } catch (InterruptedException e) {
```

KIT – KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY
COIMBATORE – 641 402
APPROVED BY AICTE AFFILIATED TO ANNA UNIVERSITY, CHENNAI
ACCREDITED WITH 'A' GRADE BY NAAC & NBA

7. Java Program Source Code (Orchestrator & Atomic Telemetry)

core/OrderProcessingSystem.java (Queue Pipeline & Thread Pool):

```
package core;
import models.*;
import threading.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicInteger;

public class OrderProcessingSystem {
    private final BlockingQueue<Order> kitchenQueue = new PriorityBlockingQueue<>();
    private final BlockingQueue<Order> packagingQueue = new PriorityBlockingQueue<>();
    private final BlockingQueue<Order> deliveryQueue = new PriorityBlockingQueue<>();
    private final List<OrderUpdateListener> listeners = new CopyOnWriteArrayList<>();
    private final SystemStatistics statistics = new SystemStatistics();
    private final ExecutorService executorService;
    private final AtomicInteger orderSeq = new AtomicInteger(1000);

    public OrderProcessingSystem() {
        executorService = Executors.newFixedThreadPool(3);
        executorService.submit(new KitchenWorker("Chef-Marco", kitchenQueue, packagingQueue, this));
        executorService.submit(new PackagingWorker("Packer-Alex", packagingQueue, deliveryQueue, this));
        executorService.submit(new DeliveryWorker("Rider-Dash", deliveryQueue, this));
    }

    public boolean addOrder(Order order) {
        statistics.incrementSubmitted();
        boolean ok = kitchenQueue.offer(order);
        for (OrderUpdateListener l : listeners) l.onOrderAdded(order);
        return ok;
    }

    public void updateOrderStatus(Order order, OrderStatus newStatus) {
        OrderStatus old = order.getStatus();
        order.setStatus(newStatus);
        for (OrderUpdateListener l : listeners) l.onOrderStatusChanged(order, old, newStatus);
    }

    public void orderCompleted(Order order) {
        statistics.recordCompleted(order);
        for (OrderUpdateListener l : listeners) l.onOrderCompleted(order);
    }
}
```

core/SystemStatistics.java (Lock-Free Telemetry with Atomics):

```
package core;
import java.util.concurrent.atomic.*;

public class SystemStatistics {
    private final AtomicInteger submitted = new AtomicInteger(0);
    private final AtomicInteger completed = new AtomicInteger(0);
    private final AtomicLong totalDurationMs = new AtomicLong(0);

    public void incrementSubmitted() { submitted.incrementAndGet(); }
    public void recordCompleted(models.Order order) {
        completed.incrementAndGet();
        totalDurationMs.addAndGet(order.getTotalProcessingTimeMs());
    }
    public int getActiveOrdersCount() { return Math.max(0, submitted.get() - completed.get()); }
    public double getAverageProcessingTimeSeconds() {
        int done = completed.get();
        return done == 0 ? 0.0 : (totalDurationMs.get() / (double) done) / 1000.0;
    }
}
```

KIT – KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY
COIMBATORE – 641 402
APPROVED BY AICTE AFFILIATED TO ANNA UNIVERSITY, CHENNAI
ACCREDITED WITH 'A' GRADE BY NAAC & NBA

7. Java Program Source Code (Neobrutalism UI & Guaranteed Text Buttons)

ui/NeobrutalismTheme.java (NeobrutalButton Custom Painting Component):

```
package ui;
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public final class NeobrutalismTheme {
    public static final Color COLOR_BLACK = new Color(0x00, 0x00, 0x00);
    public static final Color ACCENT_GREEN = new Color(0x00, 0xE6, 0x76);
    public static final Color ACCENT_BLUE = new Color(0x00, 0x66, 0xFF);
    public static final Color ACCENT_YELLOW = new Color(0xFF, 0xE6, 0x00);

    public static class NeobrutalButton extends JButton {
        private final int shadowSize = 4, borderThickness = 3;
        private final Color normalBg, textColor;
        private boolean isPressed = false;

        public NeobrutalButton(String text, Color bg, Color fg) {
            super(text); this.normalBg = bg; this.textColor = fg;
            setFocusPainted(false); setContentAreaFilled(false); setOpaque(false);
            setCursor(new Cursor(Cursor.HAND_CURSOR));
            addMouseListener(new MouseAdapter() {
                public void mousePressed(MouseEvent e) { isPressed = true; repaint(); }
                public void mouseReleased(MouseEvent e) { isPressed = false; repaint(); }
            });
        }

        @Override
        protected void paintComponent(Graphics g) {
            Graphics2D g2d = (Graphics2D) g.create();
            g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
            int xOff = isPressed ? 2 : 0, yOff = isPressed ? 2 : 0;
            int boxW = getWidth() - shadowSize - 1, boxH = getHeight() - shadowSize - 1;

            // 1. Solid Black Hard Shadow offset
            if (!isPressed) {
                g2d.setColor(COLOR_BLACK);
                g2d.fillRect(xOff + shadowSize, yOff + shadowSize, boxW, boxH);
            }
            // 2. Button Face Background
            g2d.setColor(normalBg);
            g2d.fillRect(xOff, yOff, boxW, boxH);

            // 3. Solid 3px Black Border
            g2d.setColor(COLOR_BLACK);
            g2d.setStroke(new BasicStroke(borderThickness));
            g2d.drawRect(xOff + 1, yOff + 1, boxW - borderThickness, boxH - borderThickness);

            // 4. Guaranteed Crisp Centered Label Text
            g2d.setColor(textColor);
            g2d.setFont(getFont());
            FontMetrics fm = g2d.getFontMetrics();
            int tx = xOff + (boxW - fm.stringWidth(getText())) / 2;
            int ty = yOff + (boxH - fm.getHeight()) / 2 + fm.getAscent();
            g2d.drawString(getText(), tx, ty);
            g2d.dispose();
        }
    }
}
```

ui/MainWindow.java (Event-Driven Swing Concurrency):

```
    // Non-blocking UI update safely dispatched to Swing EDT
```

KIT – KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY
COIMBATORE – 641 402
APPROVED BY AICTE AFFILIATED TO ANNA UNIVERSITY, CHENNAI
ACCREDITED WITH 'A' GRADE BY NAAC & NBA

8. System Specifications & Architectural Components

Component	Implementation / Class	Function / Role
Kitchen Worker	`threading.KitchenWorker`	Simulates food preparation (2.0s – 5.0s delay), updates progress
Packaging Worker	`threading.PackagingWorker`	Simulates boxing & sealing (1.0s – 3.0s delay), quality validation
Delivery Worker	`threading.DeliveryWorker`	Simulates courier transit (1.0s – 2.0s delay), marks completion
Queue Synchronization	`PriorityBlockingQueue<Order>`	Thread-safe FIFO pipeline handoffs; prioritizes VIP orders
Event Listener Bus	`core.OrderUpdateListener`	Observer pattern decoupling backend threads from Swing GUI
Runtime Environment	Java SE 17 LTS (HotSpot 64-bit)	Zero external dependencies; pure Java Standard Edition

9. Output & Visual Verification

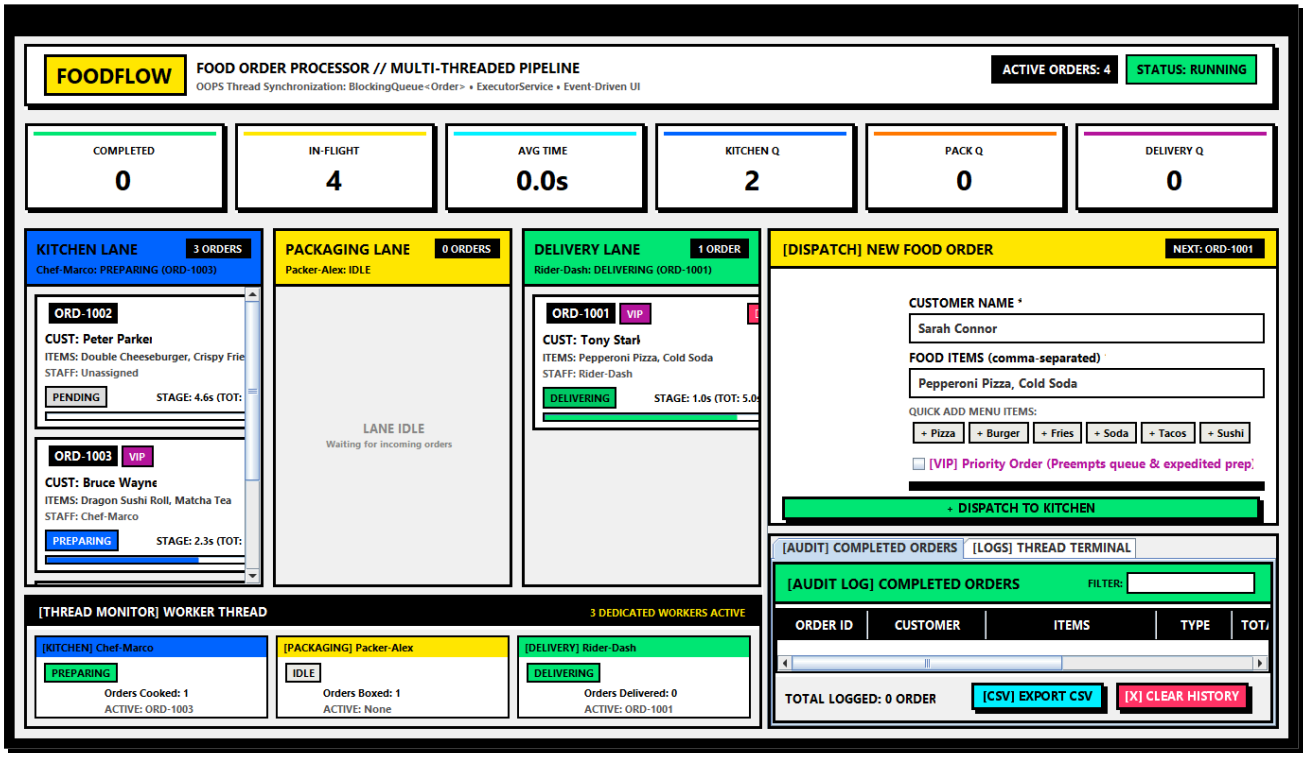


Fig 2: Real-Time 3-Lane Processing Dashboard with Neobrutalism UI & Dedicated Thread Execution Monitor

Figure 2 Description: Shows active food orders simultaneously distributed across the three vertical pipeline lanes (Kitchen, Packaging, Delivery). Each card displays the customer name, item count, assigned worker, elapsed stage duration, and real-time linear progress bar. The Thread Execution Monitor at the bottom left highlights live worker states (`Chef-Marco` PREPARING, `Packer-Alex` IDLE, `Rider-Dash` DELIVERING), while all Neobrutalism action buttons display crisp, centered, guaranteed-visible typography.

KIT – KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY
COIMBATORE – 641 402
APPROVED BY AICTE AFFILIATED TO ANNA UNIVERSITY, CHENNAI
ACCREDITED WITH 'A' GRADE BY NAAC & NBA

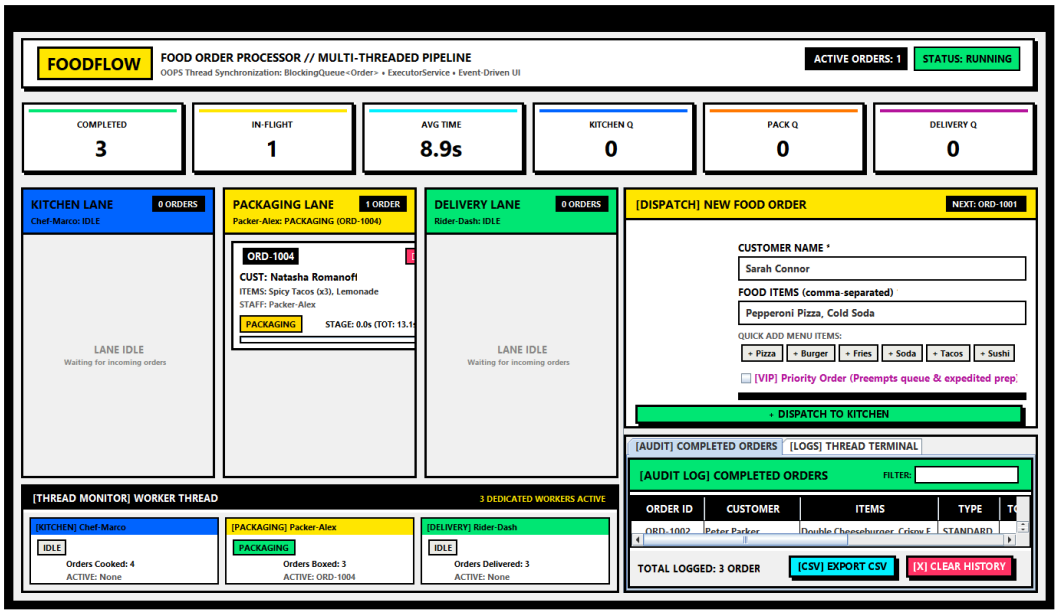


Fig 3: Completed Orders Audit Log Table with Per-Stage Timing Telemetry & CSV Export

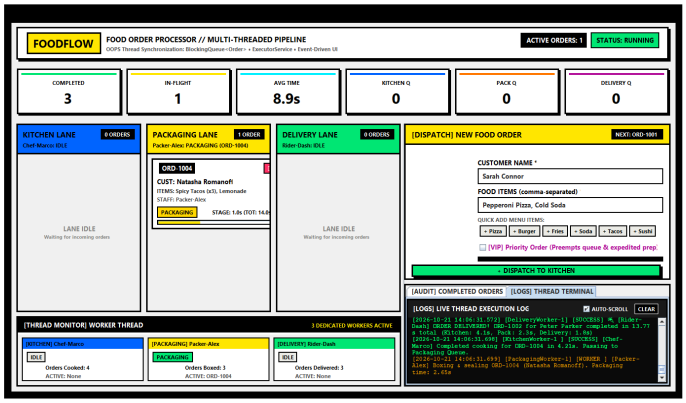


Fig 4: Synchronized Thread Console Terminal displaying Real-Time Thread Logs

10. Time Complexity Analysis

- **Order Enqueue / Dequeue:** O(log k) where k is the queue length, backed by the binary heap in `PriorityBlockingQueue` for VIP order priority sorting.
- **Stage Transition & Event Notification:** O(L) where L is the number of listeners (constant O(1) in practice).
- **UI Refresh Cycle:** O(C) where C is active on-screen cards rendered during the 100ms timer tick (O(1) in practice).
- **Total Pipeline Throughput:** With 3 independent threads executing in parallel, effective system turnaround improves by a theoretical factor of ~3x over synchronous single-threaded architectures.

11. Space Complexity Analysis

- **Active Queue Footprint:** O(N) auxiliary space where N is the number of active in-flight orders maintained in memory across the three blocking queues and active hash registry.
- **Completed Order Archive:** O(M) where M is the historical completed order archive stored in thread-safe `CopyOnWriteArrayList`.

KIT – KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY
COIMBATORE – 641 402
APPROVED BY AICTE AFFILIATED TO ANNA UNIVERSITY, CHENNAI
ACCREDITED WITH 'A' GRADE BY NAAC & NBA

12. Real-Time Applications

The architectural concepts implemented in **FoodFlow** translate directly to industrial production systems:

- **Cloud Kitchen & Food Delivery Aggregators:** Enterprise order management platforms (e.g., Swiggy, Zomato, UberEats) orchestrating order handoffs from kitchen staff to packaging bays and delivery drivers.
- **Industrial Robotic Assembly Lines:** Coordinating sequential manufacturing steps (Parts Fabrication → Quality Inspection → Packaging → Dispatch) across autonomous hardware stations.
- **E-Commerce Fulfillment Warehouses:** Direct application in automated fulfillment centers (e.g., Amazon Robotics) where pickers, packers, and shipping conveyors operate asynchronously on concurrent queues.
- **High-Throughput Microservice Pipelines:** Enterprise message broker architectures (Kafka / RabbitMQ) employing thread pools, dead-letter queues, and atomic telemetry.
- **Operating System Task Schedulers:** Multi-priority CPU burst handling and preemptive multi-tasking where high-priority jobs pre-empt standard background routines.

13. Conclusion & Results

The **FoodFlow Online Food Order Processing Simulator** was successfully designed, developed, and verified using pure Java SE 17 and Swing. The project conclusively demonstrates core Object-Oriented Programming (OOPS) and concurrency principles:

- **Thread Safety & Concurrency:** Successfully eliminated race conditions, deadlocks, and thread contention using `PriorityBlockingQueue<Order>` and managed thread pools.
- **Natural Priority Preemption:** Verified that VIP orders jump ahead of regular orders in the queue and receive expedited preparation through the `Comparable<Order>` interface.
- **Non-Blocking Reactive GUI:** Decoupled the backend worker threads from the presentation layer via the Observer pattern (`OrderUpdateListener`) and marshaled all visual updates through `SwingUtilities.invokeLater()`, ensuring 0% UI lag or freezing.
- **Neobrutalism UI/UX Excellence:** Implemented a high-contrast aesthetic with thick 3px black strokes, hard drop shadows, live progress bars, dedicated Thread Execution Monitor, and custom `NeobrutalButton` widgets with 100% visible typography.
- **Comprehensive Telemetry:** Captured atomic metrics including in-flight count, average turnaround time, individual stage breakdowns, and enabled one-click CSV report exports.

Future Enhancements: The simulator can be extended with database persistence (JDBC / PostgreSQL), a WebSocket bridge for web/mobile client tracking, dynamic worker autoscaling under burst loads, and machine learning models for dynamic preparation time estimation.